[Elbauenpark Magdeburg]

Can SAT Solvers Keep Up With the Linux Kernel's Feature Model?

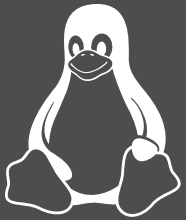
ICSE 2026 — April 12–18 — Rio de Janeiro, Brazil

Elias Kuiter[1], Urs-Benedict Braun[1], Thomas Thüm[2], Sebastian Krieter[2], Gunter Saake[1]

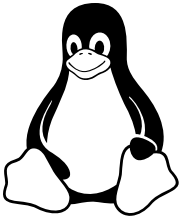
University of Magdeburg[1], Braunschweig[2], Germany

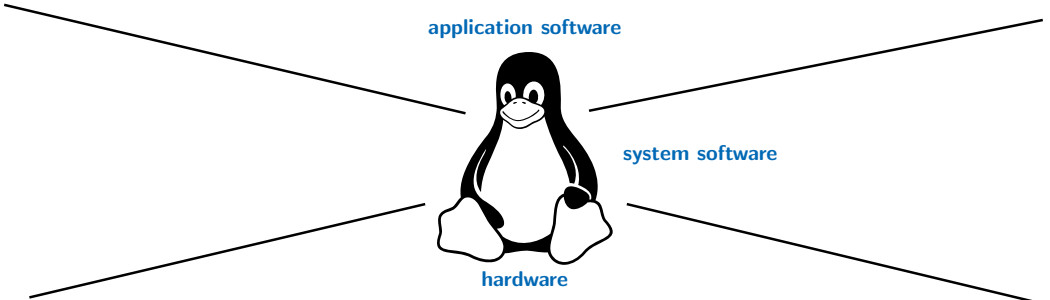
OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG

Technische Universität Braunschweig

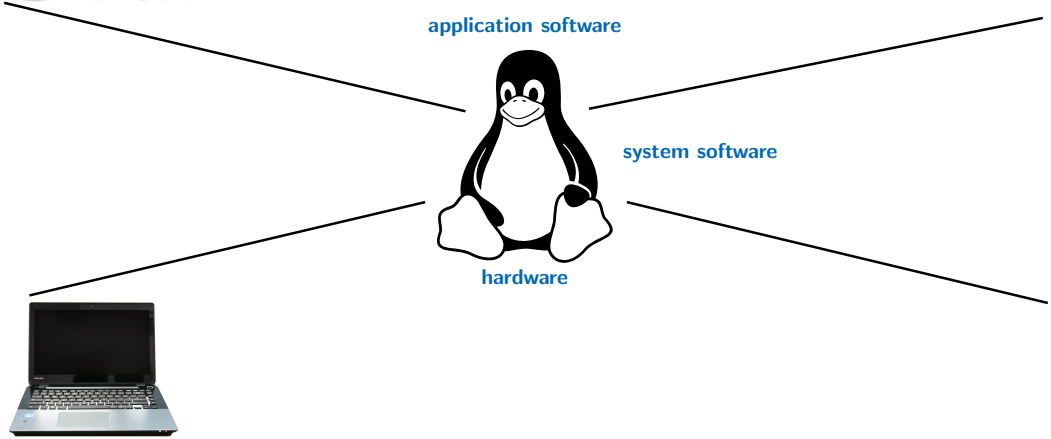


Motivation and Background

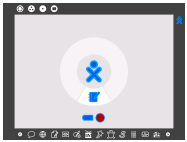



application software
system software
hardware

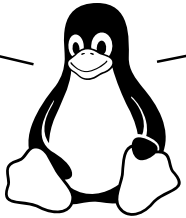
	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



application software

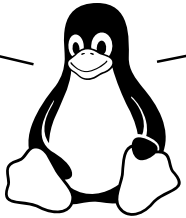


system software

hardware

	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963

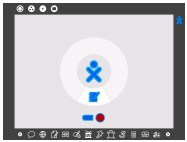
application software

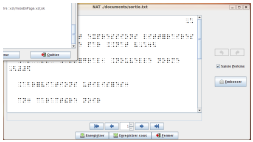


system software

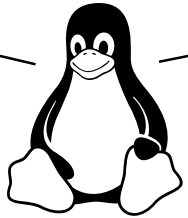
hardware

	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963





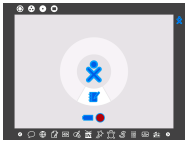
application software

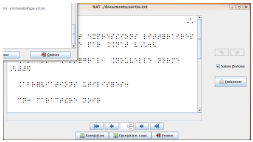


system software

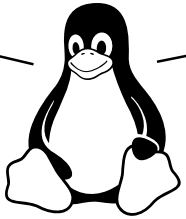
hardware

	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963





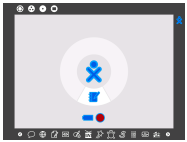
application software

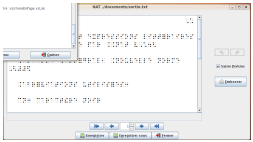


system software

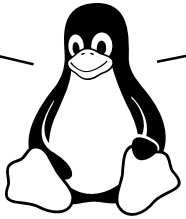
hardware

	A	B	C
1	Código	Nombre	Fecha Nacimiento
2	W966	Carlos Macías	11/24/1974
3	J925	Alex Méndez	2/23/1962
4	X326	Alberto Morales	8/24/1965
5	Q694	Julio Jiménez	8/3/1974
6	J135	Daniel Morante	11/6/1968
7	L058	Javier Carrillo	7/2/1963
8	W382	Fabián Murillo	6/25/1963
9	R186	Elías Pihuave	2/16/1967
10	F470	Iván Mendieta	7/9/1963



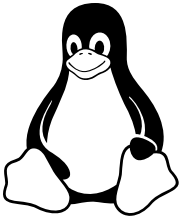


application software



system software

hardware



KConfig Feature Model ...

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

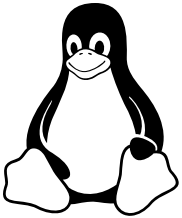
```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping



KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

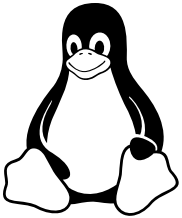
```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles



KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

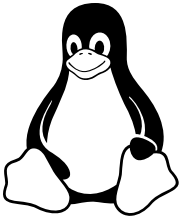
```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    tosh_smm(&regs);
#endif
```



KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

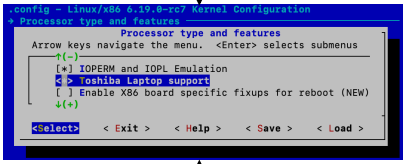
KBuild Makefiles

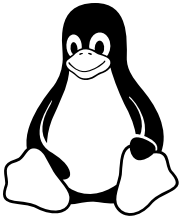
```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    tosh_smm(&regs);
#endif
```

make menuconfig





KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

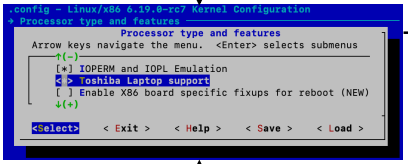
KBuild Makefiles

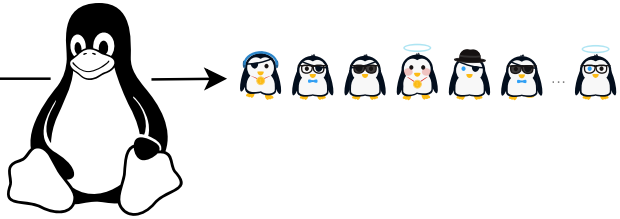
```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    tosh_smm(&regs);
#endif
```

make menuconfig





KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

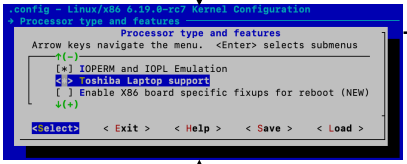
KBuild Makefiles

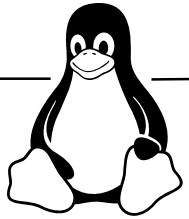
```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    tosh_smm(&regs);
#endif
```

make menuconfig





Linux is a

software product line

KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

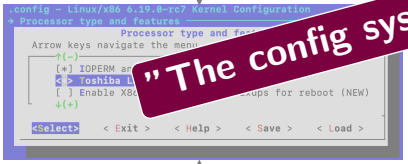
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    _smm(&regs);
```

make menuconfig



"The config system is a nightmare"

Linux is a
software product line

KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

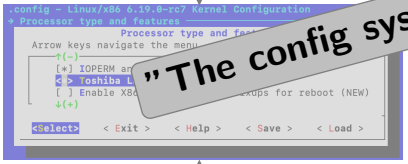
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    _smm(&regs);
```

make menuconfig



"The config system is a nightmare"

"It's gotten out of control"

Linux is a

software product line

KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

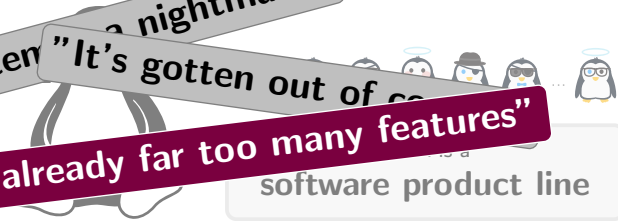
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    _smm(&regs);
```

make menuconfig



"The config system is a nightmare"

"It's gotten out of control"

"There are simply already far too many features"

software product line

KConfig Feature Model

```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```

Feature-to-Code Mapping

```
obj-$(CONFIG_TOSHIBA) += toshiba.o
```

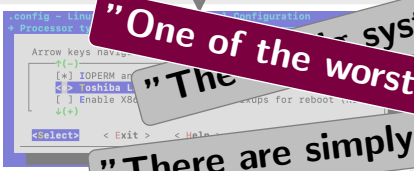
KBuild Makefiles

```
#ifdef CONFIG_TOSHIBA
#include <linux/toshiba.h>
#endif
```

C Preprocessor

```
#ifdef CONFIG_TOSHIBA
    SMMRegisters regs;
    regs.eax = 0xff00; /* HCI_SET */
    regs.ebx = 0x0002; /* HCI_BACKLIGHT */
    regs.ecx = 0x0000; /* HCI_DISABLE */
    _smm(&regs);
```

make menuconfig



"One of the worst parts of the whole project" [Linus Torvalds]

"There are simply alre… …so many features"

"It's gotten out of co…"

"…a nightmare"

"…d system…"

"The…"

…product line

KConfig Feature Model

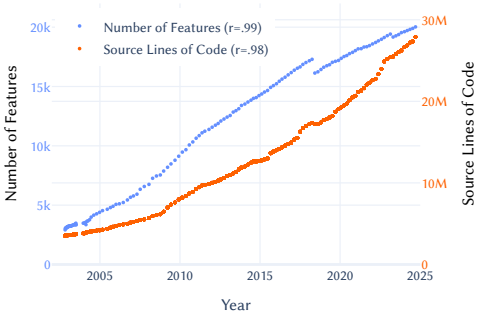
```
config TOSHIBA
  tristate "Toshiba Laptop"
  depends on X86_32
  help
    Toshiba portables with a
    genuine Toshiba BIOS
```

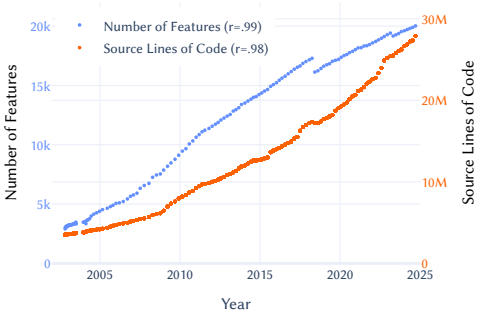
```
config OLPC
  bool "One Laptop Per Child"
  depends on !X86_PAE
  help
    Unique features of the OLPC
    XO hardware
```

```
config SUPERH
  def_bool y
  select ARCH_32BIT_OFF
  help
    Sega Dreamcast gaming
    console
```

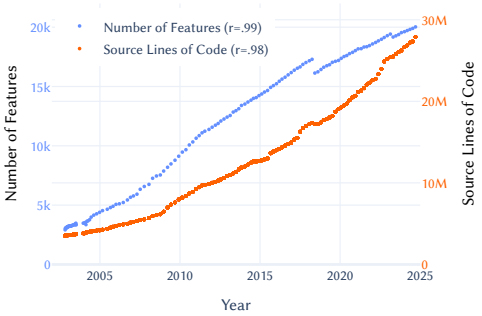
```
config X86_32_IRIS
  tristate "Eurobraille/Iris"
  depends on X86_32
  help
    Iris machines from
    EuroBraille
```

```
config X86_GOLDFISH
  bool "Goldfish"
  depends on X86_EXT
  help
    Goldfish virtual platform
    for Android development
```


Number of Features (r=.99)
Source Lines of Code (r=.98)
Number of Features
Source Lines of Code
Year
20k
15k
10k
5k
0
30M
20M
10M
0
2005
2010
2015
2020
2025

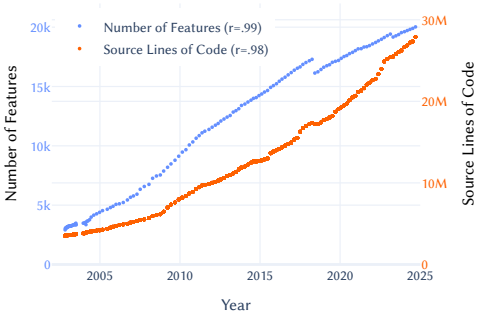


The Linux kernel grows **more complex** ...



interactive product configuration anomaly detection and explanation feature-model evolution and slicing modularization implicit constraints code parsing dead code analysis code simplification type checking model checking formal verification static code analysis dataflow analysis testing t-wise sampling uniform random sampling optimization of non-functional properties refactoring economical estimations ...

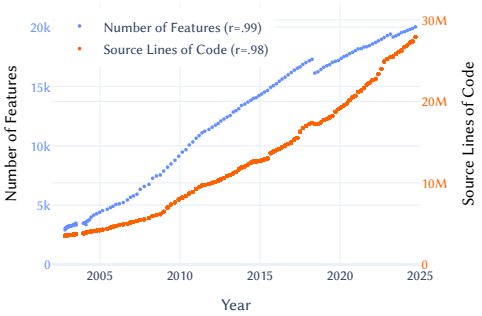
The Linux kernel grows **more complex** ...



interactive product configuration anomaly detection and explanation feature-model evolution and slicing modularization implicit constraints code parsing dead code analysis code simplification type checking model checking formal verification static code analysis dataflow analysis testing t-wise sampling uniform random sampling optimization of non-functional properties refactoring economical estimations ...

The Linux kernel grows **more complex** ...

... and so do **dozens of analyses**.



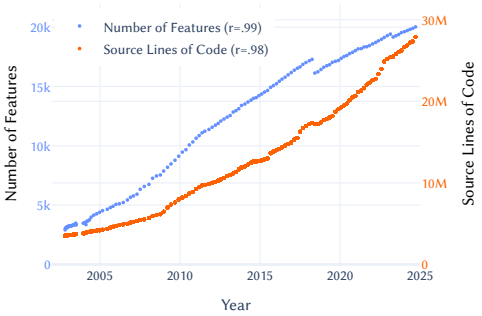

```
config TOSHIBA
 tristate "Toshiba Laptop"
 depends on X86_32
 help
  Toshiba portables
  with a genuine Toshiba BIOS
```

checking formal verification static code analysis dataflow analysis testing t-wise sampling uniform random sampling optimization of non-functional properties refactoring economical estimations ...

The Linux kernel grows **more complex** ...

... and so do **dozens of analyses** ...

... relying on **feature models** ...




```
config TOSHIBA
 tristate "Toshiba Laptop"
 depends on X86_32
 help
  Toshiba portables
  with a genuine Toshiba BIOS
```

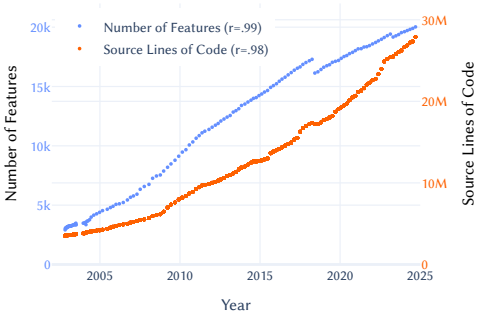
checking static code analysis testing t-wise sampling sampling optimization of properties refactoring ...

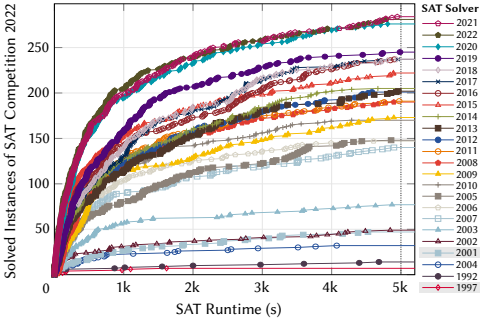
The Linux kernel grows **more complex** ...

... and so do **dozens of analyses** ...

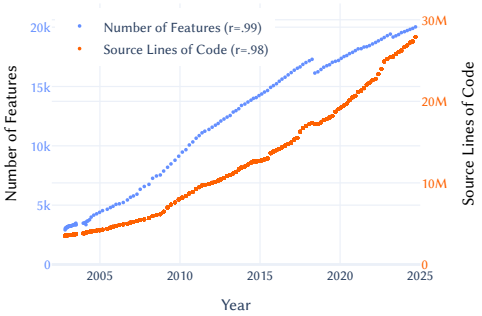
... relying on **feature models** ...

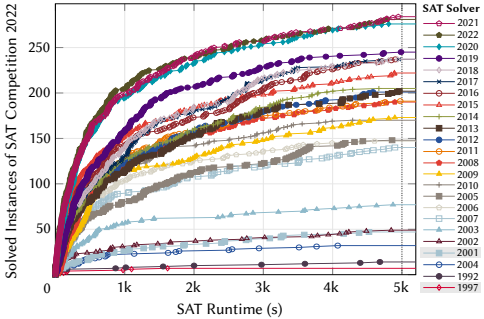
... relying on **SAT solvers**.





The Linux kernel grows **more complex** . . .

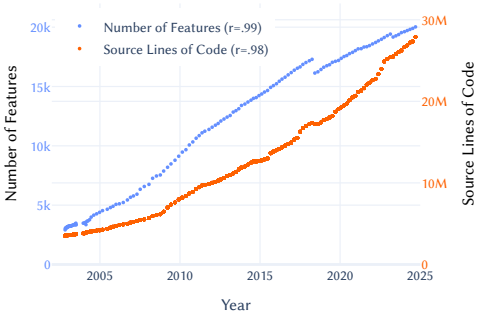




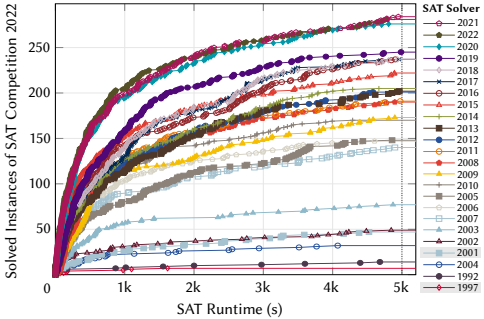
The Linux kernel grows **more complex** ...

... yet SAT solvers **get faster** every year.

Linux Kernel vs. SAT Solvers



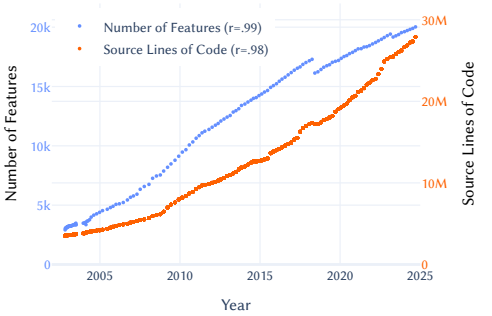
vs.



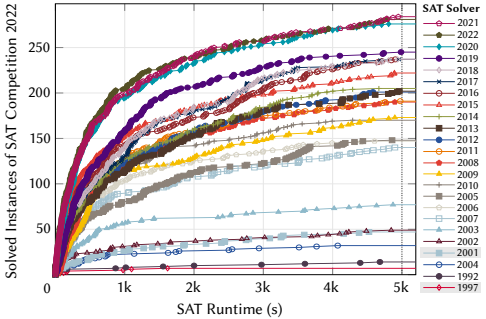
The Linux kernel grows **more complex** ...

... yet SAT solvers **get faster** every year.

Linux Kernel vs. SAT Solvers



vs.

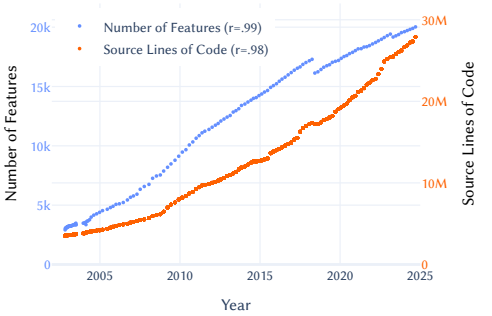


The Linux kernel grows **more complex** ...

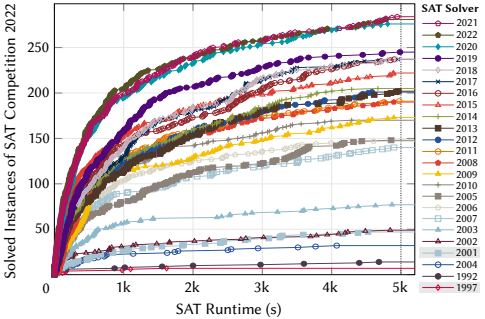
... yet SAT solvers **get faster** every year.

Can SAT solvers **keep up** with the Linux kernel?

Linux Kernel vs. SAT Solvers



vs.



The Linux kernel grows **more complex** ...

... yet SAT solvers **get faster** every year.

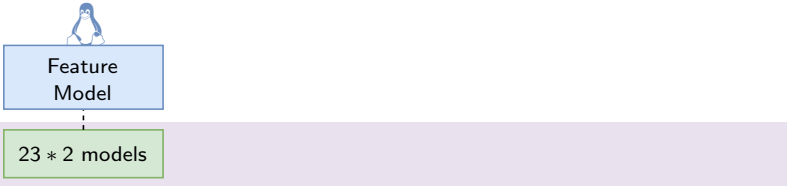
Can SAT solvers **keep up** with the Linux kernel? – Spoiler: **They cannot!**


How to evaluate this?

Research Questions and Evaluation Pipeline

Research Questions and Evaluation Pipeline

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?

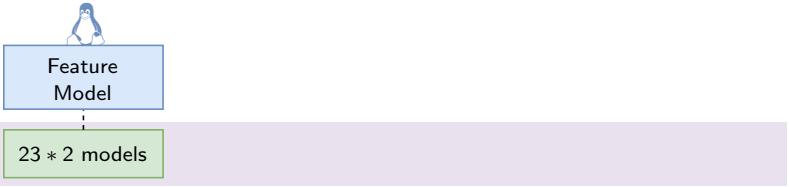


Years 2002–24

Research Questions and Evaluation Pipeline

RQ_1 **Influence** of architecture,

RQ_2 How does SAT runtime **evolve** on the history of Linux?

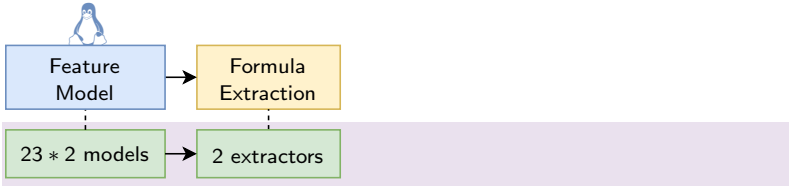


Years 2002–24
`x86` and `arm`

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor,

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?

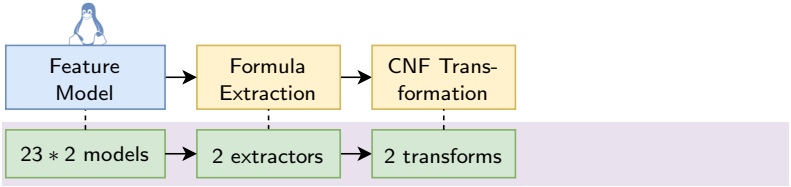


Years 2002–24	KConfigReader
`x86` and `arm`	KClause

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation,

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?

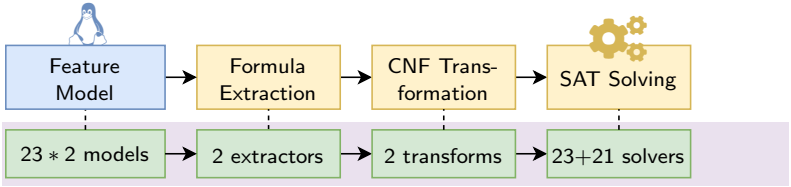


Years 2002–24	KConfigReader	KConfigReader
`x86` and `arm`	KClause	Z3

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler,

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?

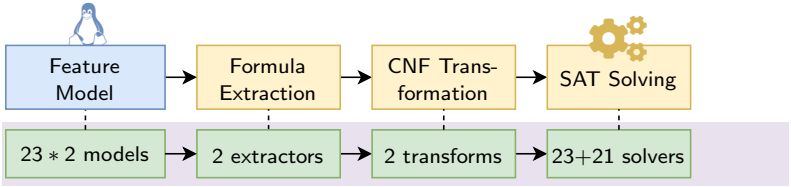


Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause	Z3	SAT Museum

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler, iterations?

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?

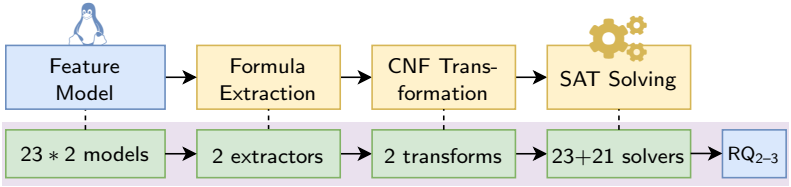


Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause ∗5	Z3 ∗5	SAT Museum ∗3

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler, iterations?

RQ$_2$ How does SAT runtime **evolve** on the history of Linux?



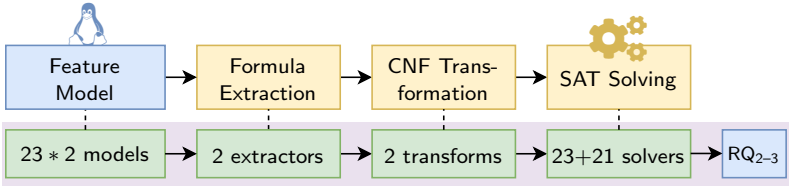
Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause ∗5	Z3 ∗5	SAT Museum ∗3

Measured Metric

Runtime of a SAT Query

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler, iterations?
RQ$_2$ How does SAT runtime **evolve** on the history of Linux?
RQ$_3$ How do **optimal solvers** evolve (e.g., virtual best solver)? **RQ$_4$** And industrial solvers?



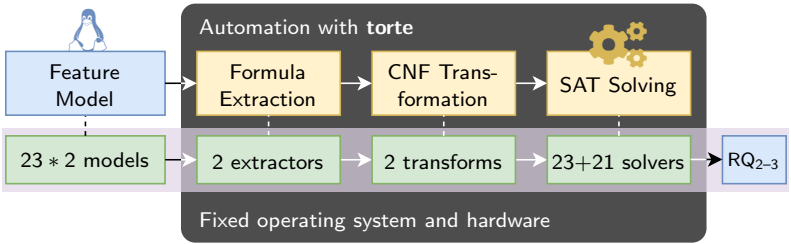
Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause ∗5	Z3 ∗5	SAT Museum ∗3

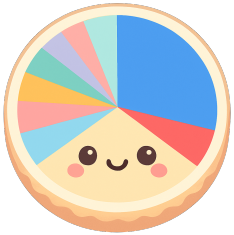
Measured Metric

Runtime of a SAT Query

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler, iterations?
RQ$_2$ How does SAT runtime **evolve** on the history of Linux?
RQ$_3$ How do **optimal solvers** evolve (e.g., virtual best solver)? **RQ$_4$** And industrial solvers?





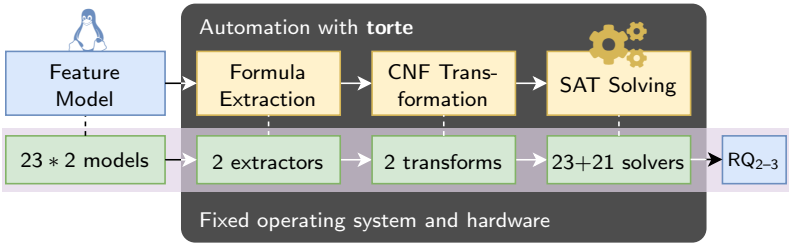
Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause ∗5	Z3 ∗5	SAT Museum ∗3

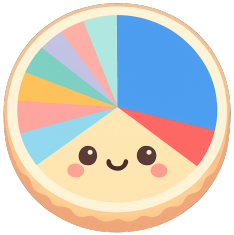
Measured Metric

Runtime of a SAT Query

Research Questions and Evaluation Pipeline

RQ$_1$ **Influence** of architecture, extractor, CNF transformation, C/C++ compiler, iterations?
RQ$_2$ How does SAT runtime **evolve** on the history of Linux?
RQ$_3$ How do **optimal solvers** evolve (e.g., virtual best solver)? **RQ$_4$** And industrial solvers?





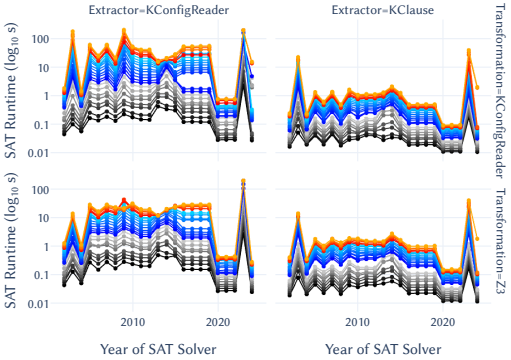
Years 2002–24	KConfigReader	KConfigReader	SAT Competition
`x86` and `arm`	KClause ∗5	Z3 ∗5	SAT Museum ∗3

Measured Metric

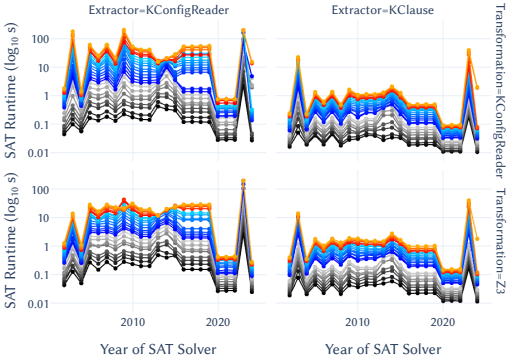
Runtime of a SAT Query

RQ_2: How Does ⚙ SAT Runtime Evolve on 🐧 Linux?

RQ_2: How Does SAT Runtime Evolve on Linux? (logarithmic y-axis!)

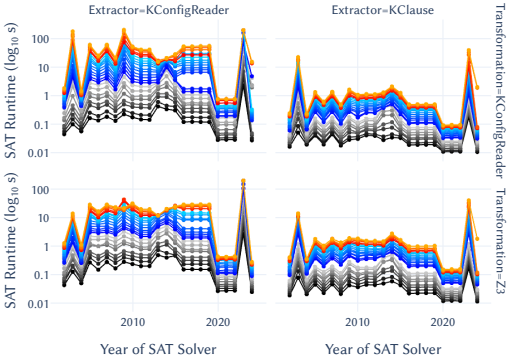


RQ_2: How Does ⚙ SAT Runtime Evolve on 🐧 Linux? (logarithmic y-axis!)



Every Line Is a 🐧 Feature Model . . .

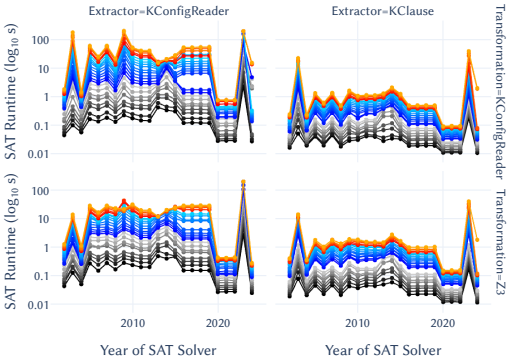
RQ_2: How Does ⚙ SAT Runtime Evolve on 🐧 Linux? (logarithmic y-axis!)

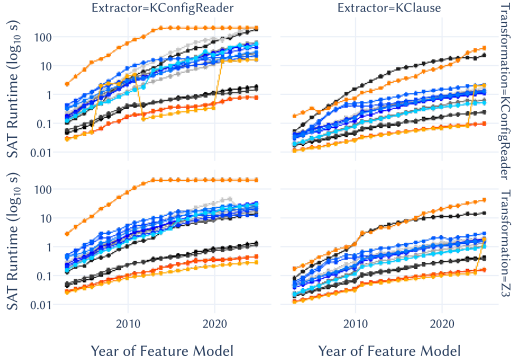


Every Line Is a 🐧 Feature Model . . .

. . . all of which stagnate in 2005–19.

RQ_2: How Does SAT Runtime Evolve on Linux? (logarithmic y-axis!)

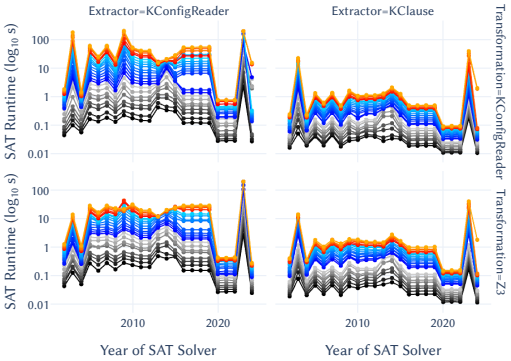


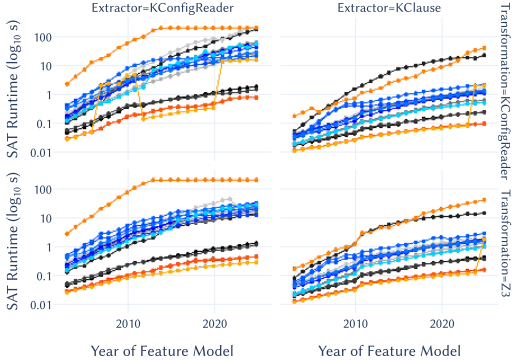


Every Line Is a Feature Model ...

... all of which stagnate in 2005–19.

RQ_2: How Does SAT Runtime Evolve on Linux? (logarithmic y-axis!)



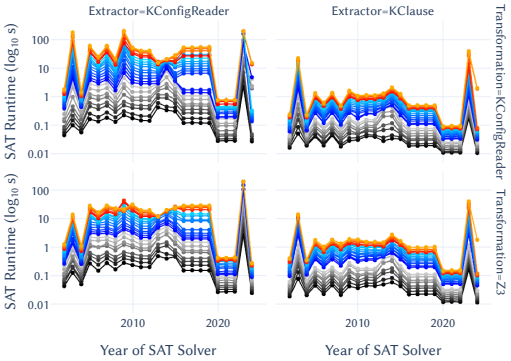


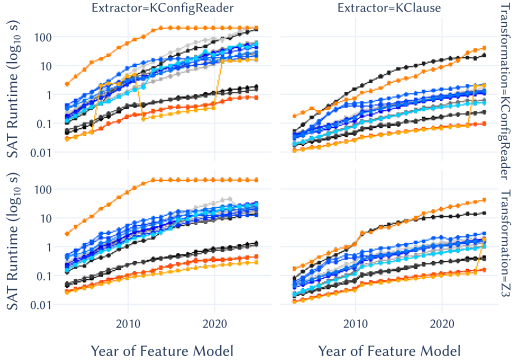
Every Line Is a Feature Model ...

... all of which stagnate in 2005–19.

Every Line Is a SAT Solver ...

RQ_2: How Does SAT Runtime Evolve on Linux? (logarithmic y-axis!)





Every Line Is a Feature Model ...

... all of which stagnate in 2005–19.

Every Line Is a SAT Solver ...

... all of which slow down over time.

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

Let's "Travel Back in Time" ...

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

Let's "Travel Back in Time" ...

How fast was the best solver of year x on Linux of year x?

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

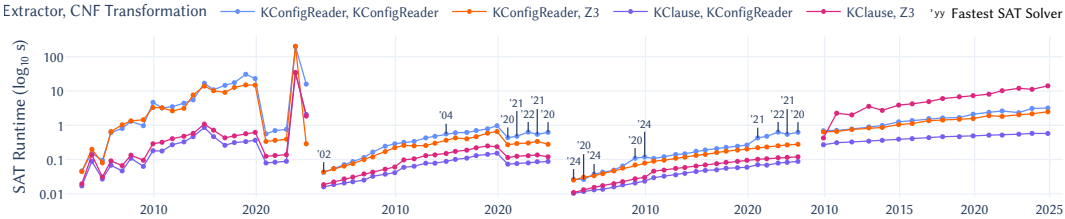
Let's "Travel Back in Time" ...

How fast was the best solver of year x on Linux of year x? (+ variations)

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

Let's "Travel Back in Time" ...

How fast was the best solver of year x on Linux of year x? (+ variations)

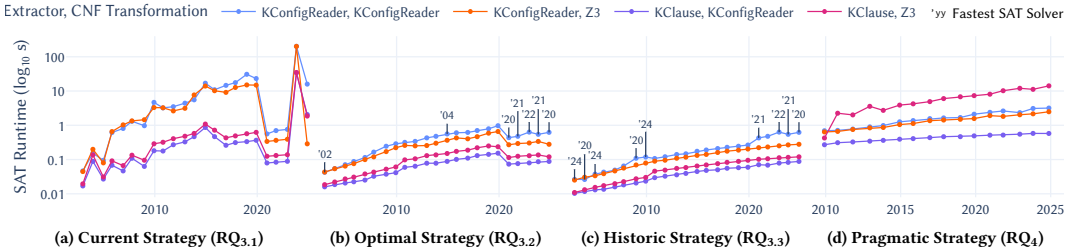


(a) Current Strategy ($RQ_{3.1}$) (b) Optimal Strategy ($RQ_{3.2}$) (c) Historic Strategy ($RQ_{3.3}$) (d) Pragmatic Strategy (RQ_4)

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

Let's "Travel Back in Time" ...

How fast was the best solver of year x on Linux of year x? (+ variations)



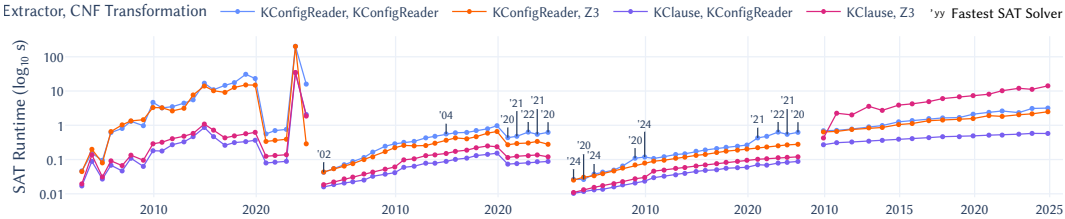
(a) Current Strategy ($RQ_{3.1}$) (b) Optimal Strategy ($RQ_{3.2}$) (c) Historic Strategy ($RQ_{3.3}$) (d) Pragmatic Strategy (RQ_4)

All slow down by 10%–16% per year. Performance halves **every seven years**.

RQ_3: How Do Optimal ⚙ Solvers Evolve on 🐧 Linux?

Let's "Travel Back in Time" ...

How fast was the best solver of year x on Linux of year x? (+ variations)



(a) Current Strategy ($RQ_{3.1}$) (b) Optimal Strategy ($RQ_{3.2}$) (c) Historic Strategy ($RQ_{3.3}$) (d) Pragmatic Strategy (RQ_4)

All slow down by 10%–16% per year. Performance halves **every seven years**.*

* despite disruptive innovations like `Kissat`

Conclusion

Conclusion

The Linux kernel grows **more complex**.

SAT solvers **get faster** every year.

Conclusion

The Linux kernel grows **more complex**.

SAT solvers **get faster** every year.

SAT solvers cannot **keep up** with the Linux kernel's feature model.

Conclusion

The Linux kernel grows **more complex**.

SAT solvers **get faster** every year.

SAT solvers <u>cannot</u> **keep up** with the Linux kernel's feature model.

Conclusion

The Linux kernel grows **more complex**.

SAT solvers **get faster** every year.

SAT solvers cannot **keep up** with the Linux kernel's feature model.

A Cautionary Tale

- Will we be able to analyze the Linux kernel **in 10–20 years**?
  variability bugs may proliferate!
- Can SAT solvers keep up with **other system software**?

Conclusion

The Linux kernel grows **more complex**. *vs.* SAT solvers **get faster** every year.

SAT solvers cannot **keep up** with the Linux kernel's feature model.

A Cautionary Tale

- Will we be able to analyze the Linux kernel **in 10–20 years**?
  variability bugs may proliferate!
- Can SAT solvers keep up with **other system software**?

Mitigations

- (hope for hardware → Moore's law)
- **kernel developers**: reduce variability or limit growth
- **automated reasoning**: improve SAT solvers + benchmarks
- **SE researchers**: apply IPASIR + compositional analyses

Conclusion

The Linux kernel grows **more complex**.

vs.

SAT solvers **get faster** every year.

SAT solvers cannot **keep up** with the Linux kernel's feature model.

A Cautionary Tale

- Will we be able to analyze the Linux kernel **in 10–20 years**?
  variability bugs may proliferate!
- Can SAT solvers keep up with **other system software**?

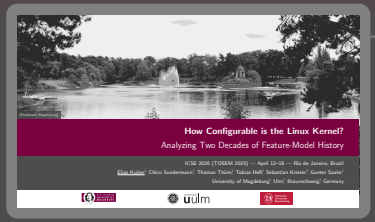
Mitigations

- (hope for hardware → Moore's law)
- **kernel developers**: reduce variability or limit growth
- **automated reasoning**: improve SAT solvers + benchmarks
- **SE researchers**: apply IPASIR + compositional analyses

Thank you for listening!

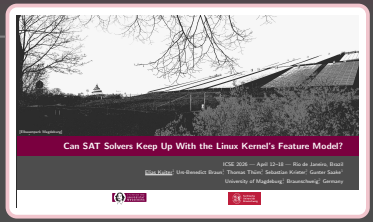
Conclusion

Journal-First Paper



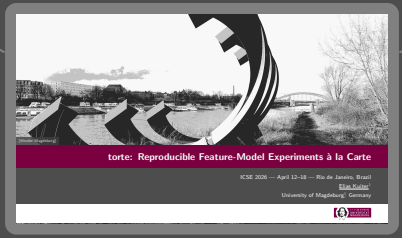
How Configurable Is the Linux Kernel?

Research Paper



Can SAT Solvers Keep Up With the Linux Kernel's Feature Model?

Demo Paper



torte: Reproducible Feature-Model Experiments à la Carte

Experiment Variables

Variable	Type	Values	RQ
Feature Model			
Version	Independent	*(cf. Table 2a)*	$RQ_{2–4}$
Architecture	Independent	`x86`, `arm`	$RQ_{1.1}$
Formula Extraction	Independent	KConfigReader, KClause	$RQ_{1.2,\ 2–4}$
CNF Transformation	Independent	KConfigReader, Z3	$RQ_{1.3,\ 2–4}$
SAT Solving			
Solver	Independent	*(cf. Table 2b)*	$RQ_{2–4}$
Compiler (only for ①)	Confounding?	Competition: `gcc/g++ ??`	$RQ_{1.4}$
	Control	Museum: `gcc/g++ 9.4.0`	
Analysis	Control	SAT query (20min timeout)	—
Deep Variability			
Operating System	Control	Ubuntu `22.04` (in Docker)	—
Hardware	Control	Intel Xeon E5-2630 2.4 GHz	—
Iteration	Independent	$5 * 5 * 3$ iterations	$RQ_{1.5,\ 2}$
SAT Runtime	Dependent	*(cf. Section 5)*	$RQ_{1–4}$

Feature Models and SAT Solvers

Year	Version	Year	Version
’02	2.5.53	’14	3.18
’03	2.6.0	’15	4.3
’04	2.6.10	’16	4.9
’05	2.6.14	’17	4.14
’06	2.6.19	’18	4.20
’07	2.6.23	’19	5.4
’08	2.6.28	’20	5.10
’09	2.6.32	’21	5.15
’10	2.6.36	’22	6.1
’11	3.1	’23	6.6
’12	3.7	’24	6.12
’13	3.12	*’25*	*N/A*

In experiment ①, we analyze all years. In experiment ②, we only analyze years ’09–’24. In each analyzed year, we consider two architectures (`x86`, `arm`). We only consider the mainline kernel of Linux.

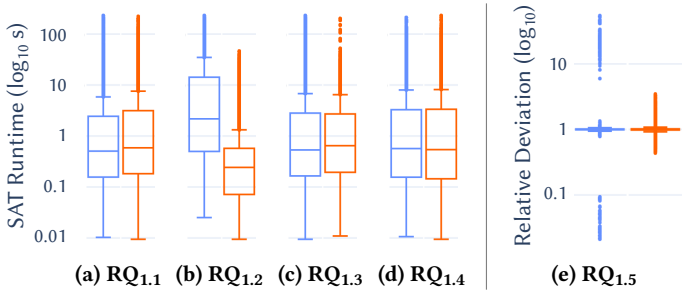
(a) Feature Models

	Year	Solver
Experiment ①	’02, ’04	`zChaff`†, `Limmat`‡
	’03	`Forklift`,† `BerkMin`‡
	’05	`SatELiteGTI`
	’06, ’08	`MiniSat`
	’07	`RSat`
	’09	`PrecoSAT`
	’10	`CryptoMiniSat`
	’11–12	`Glucose`
	’13–14	`Lingeling`
	’15	`abcdSAT`
	’16–19	`MapleSAT`
	’20–22, ’24	`Kissat` (’24†)
	’23	`SBVA-CaDiCaL`†
Exp. ②	’09–11	`Sat4j (2.1.0)`
	’11–14	`Sat4j (2.3.1)`
	≥ 2014	`Sat4j (2.3.5)`

†SAT Competition only ‡SAT Museum only

(b) SAT Solvers

RQ_1: Influence Factors



(a) $RQ_{1.1}$ (b) $RQ_{1.2}$ (c) $RQ_{1.3}$ (d) $RQ_{1.4}$ (e) $RQ_{1.5}$

Image Credits

- **Tux, the Linux penguin**, Larry Ewing and The GIMP [https://commons.wikimedia.org/wiki/File:Tux.png]
- **Microsoft Excel**, FreddyMendozaRueda [https://commons.wikimedia.org/wiki/File:Crear-tablas-en-excel-0000161.png]
- **XO Sugar**, Firefoxman [https://commons.wikimedia.org/wiki/File:XO-sugar.png]
- **Sonic Adventure**, Sonic Team [https://en.wikipedia.org/wiki/File:Sonic_Adventure_Dreamcast.png]
- **NAT Braille**, Raminala [https://commons.wikimedia.org/wiki/File:CaptureNAT.png]
- **Android 16**, Paowee [https://commons.wikimedia.org/wiki/File:Android_16_home_screen_screenshot.png]
- **Toshiba Satellite S40t**, Cheon Fong Liew [https://commons.wikimedia.org/wiki/File:Toshiba_Satellite_S40t_laptop_-_10087025595.jpg]
- **XO Laptop**, One Laptop per Child [https://commons.wikimedia.org/wiki/File:Xo_intro_v2.jpg]
- **EuroBraille Iris**, Mfaure [https://commons.wikimedia.org/wiki/File:Plage-braille-avec-touches-speciales.jpg]
- **Bugdroid**, Irina Blok [https://commons.wikimedia.org/wiki/File:Android_2023_3D_logo_and_wordmark.svg]
- **Sega Dreamcast**, Evan-Amos [https://commons.wikimedia.org/wiki/File:Dreamcast-Console-Set.jpg]
- **SAT Solver**, Chico Sundermann [https://github.com/TUBS-ISF/Slides/blob/main/2021/2021-04-14-FOSD-AnalyzingFeatureModelsWithSharpSAT.pdf]
- **TikZpingus**, Florian Sihler [https://github.com/EagleoutIce/tikzpingus]

All images were used with permission. The slides were created with FancyBeamer and draw.io.